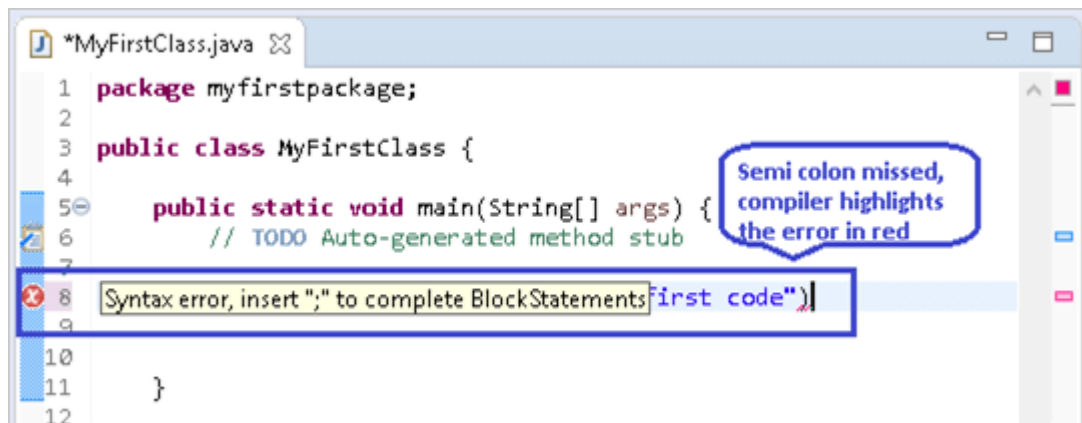


Basics Of Debugging Your Code In Eclipse

- When you start coding, you can never write perfect or even a correct code in the first attempt.
- Getting your script running perfectly sometimes requires debugging the code to get the output or the output that you are looking for.

What is Debugging?

- Debugging is a technique that is used to see your code execute line by line.
- That means you are able to see the execution of each line of your code and stop at any line of the code and analyze the code, the variables, and the values these variables carry at the time you have halted the execution.
- Debugging is used extensively to find runtime errors in your code.
- Compiler errors are highlighted in the code itself in the Eclipse IDE and with compile-time errors, you cannot proceed to run your program.



Code:

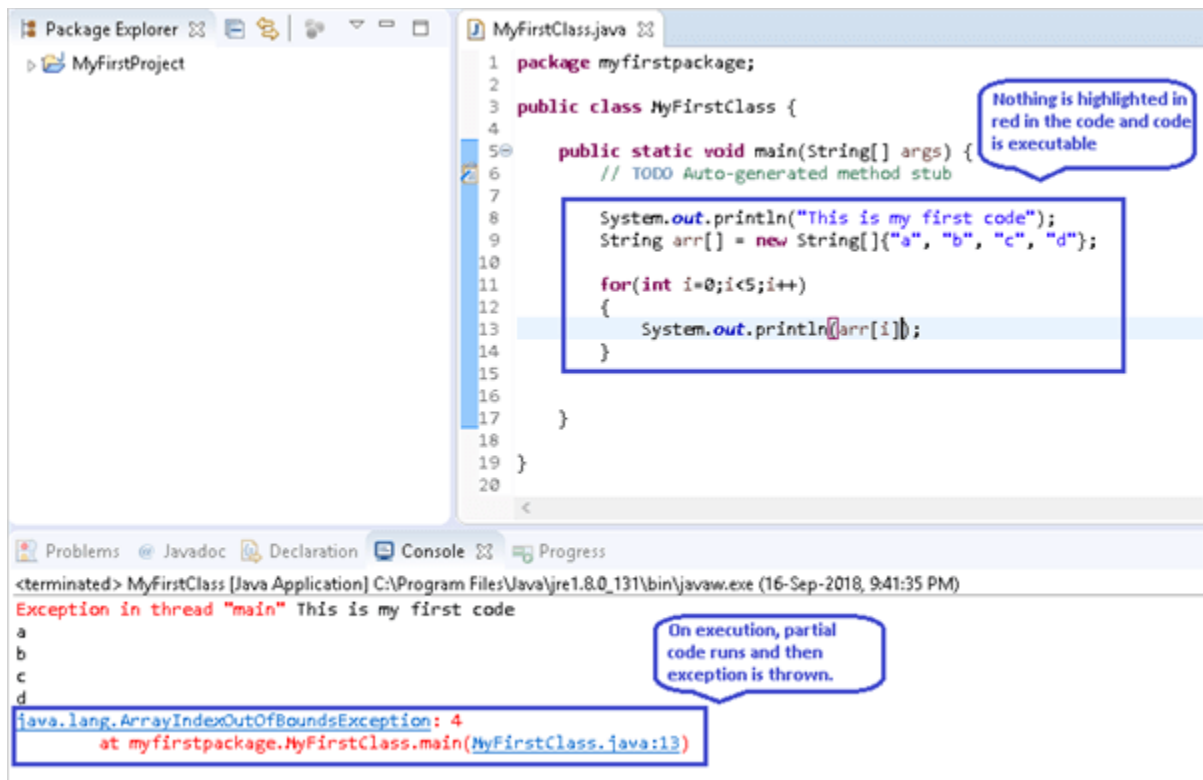
```
package myfirstpackage;

public class MyFirstClass {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        System.out.println("This is my first code")
    }
}
```

- However, if you have runtime exceptions, then they may not be highlighted in the code, instead, when you run the code, your program will fail due to this exception.



Code:

```

package myfirstpackage;

public class MyFirstClass {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        System.out.println("This is my first code");
        String arr[] = new String[]{"a", "b", "c", "d"};

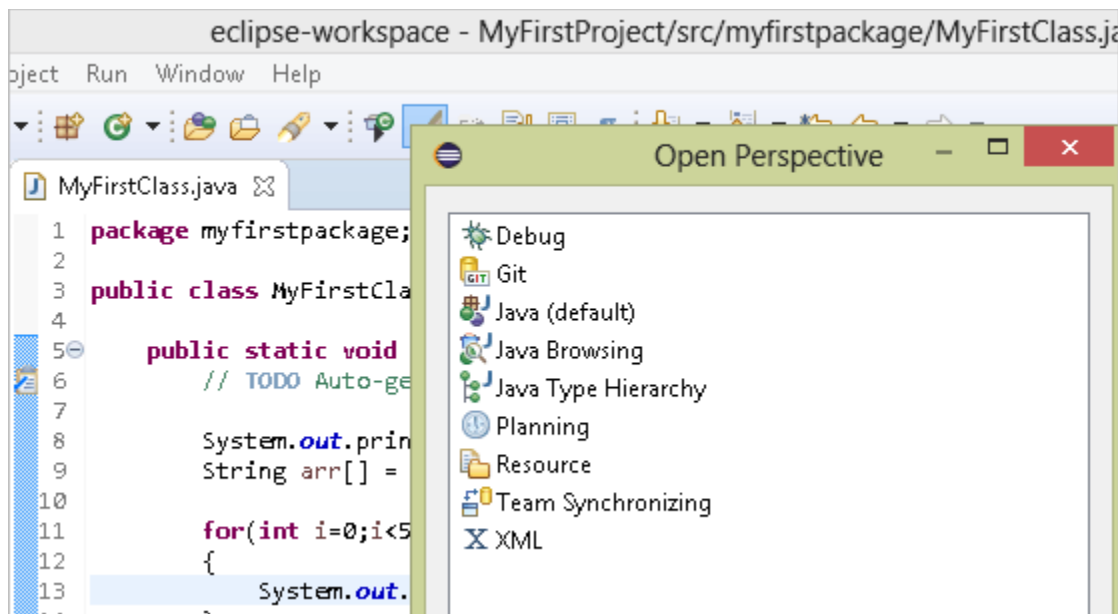
        for(int i=0;i<5;i++)
        {
            System.out.println(arr[i]);
        }
        System.out.println("This is my first code2");
    }
}

```

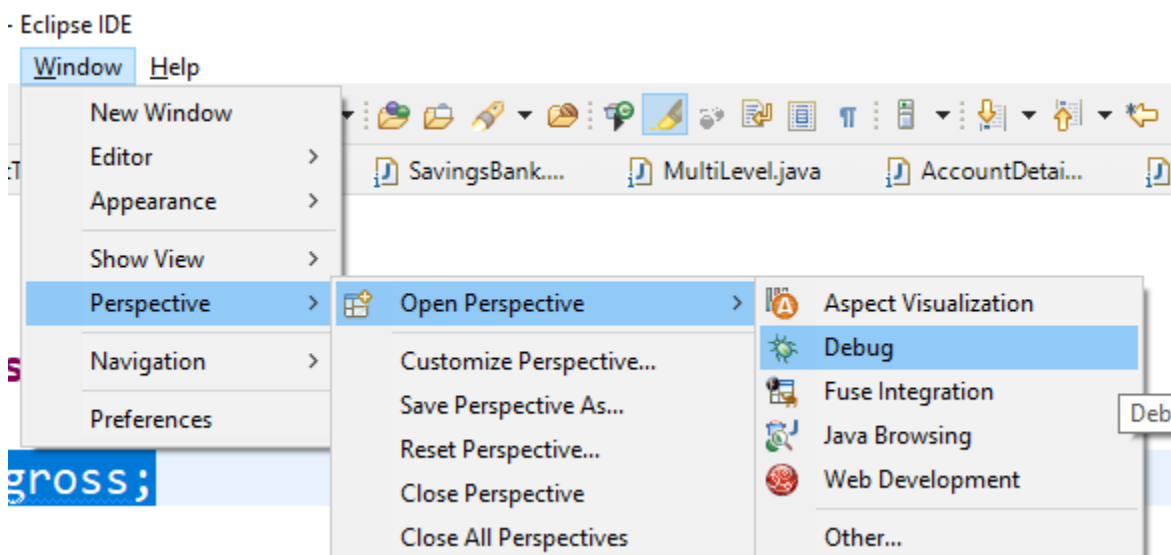
- Debugging helps for such exceptions.
- Debugging will let you see how the code enters the “for loop” for every iteration and shows you why it is throwing an error after printing a few options.
- In the example we are using, you may be able to understand the issue just by viewing the exception, as the array size is less than the number of times the for loop is running this exception is being thrown.
- However, as this guide is more about the debugger, we have chosen this simple code.
- The same logic can be applied to complex codes as well.

How to Launch the Debugger?

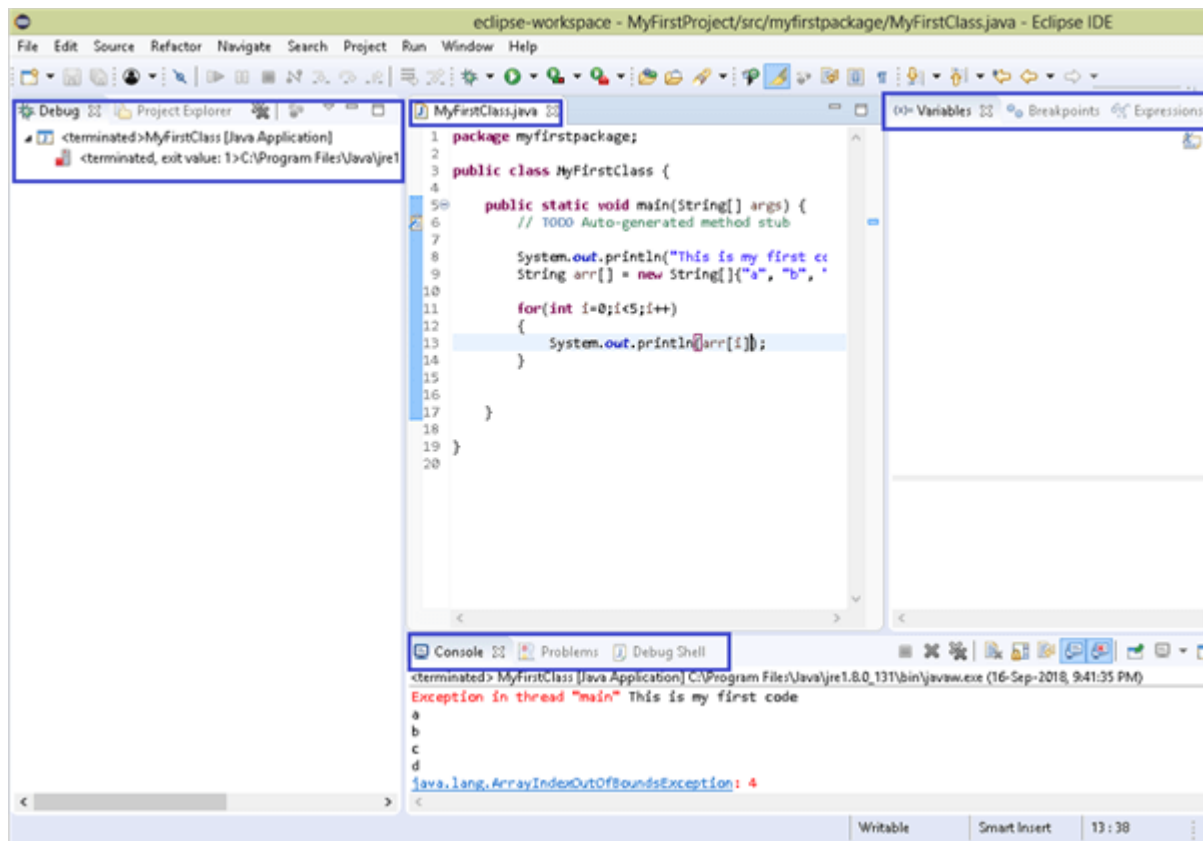
- There are multiple ways to launch the debugger on your code.
- You can launch the debugger through the perspectives in Eclipse.



- Debug is the first view available in the list of perspectives.
- Let's see what happens when you launch this perspective. Click on the first option in the "Open Perspective" and click on open.



The debug perspective will show the following window:

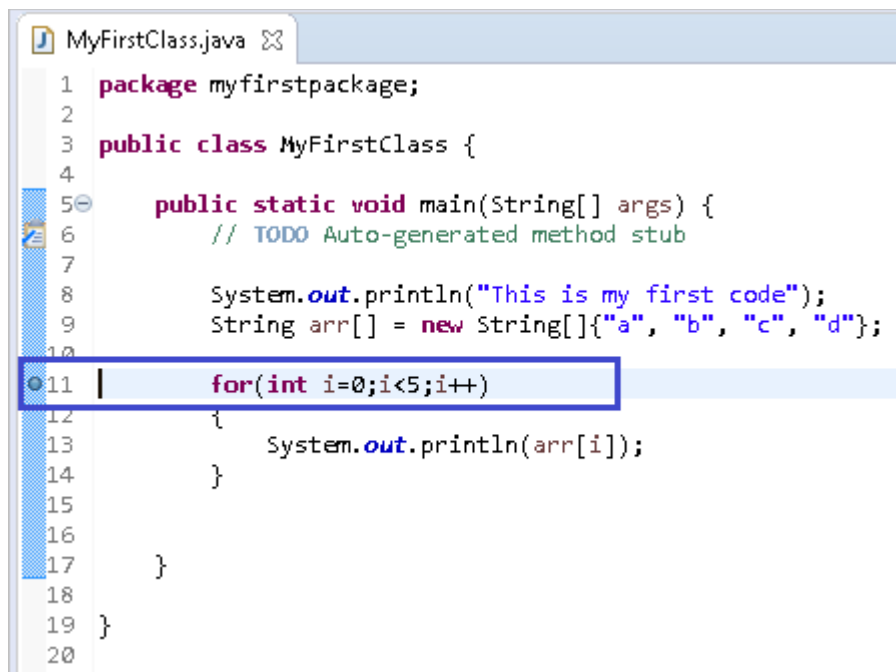


- **Debug window:** Right next to the project explorer the debug explorer is opened, in which the class being debugged is displayed.
 - **Class:** This is the class that you want to debug.
 - **Variables:** This section is where you can view the variables and how their state is changing during execution. By right-clicking on the variables displayed here, you can do multiple operations on them like change them or view their data type, etc.
 - **Breakpoint:** In this section, you can view and change breakpoints (explained further). From here you can perform advanced operations on breakpoints such as defining conditions on them.
 - **Console:** This is where you can see the execution happening.
-
- Now as we have the debugger, let's debug.
 - If you run your code directly at this point, it will run completely and halt when an exception is thrown.
 - However, you want it to halt when it enters the array and from there you want to run every line of the code manually.

The Breakpoint

- To take manual control of your program at the time of execution you need something called a breakpoint.
- A breakpoint is a way to tell the debugger from where you want to take control of your code.
- Applying a breakpoint is very simple.

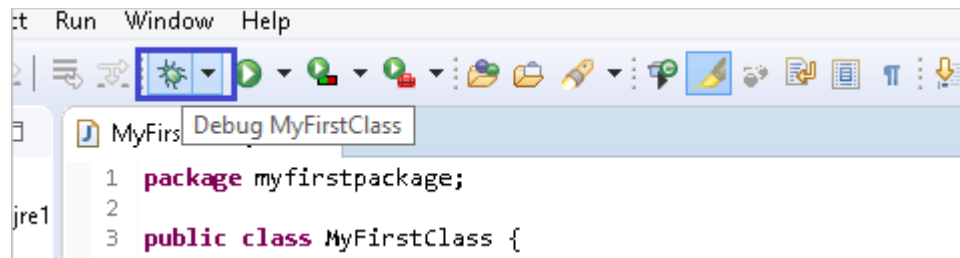
On the line where you want the breakpoint, double-click on the left margin, and a blue circle is created next to the line as shown below:



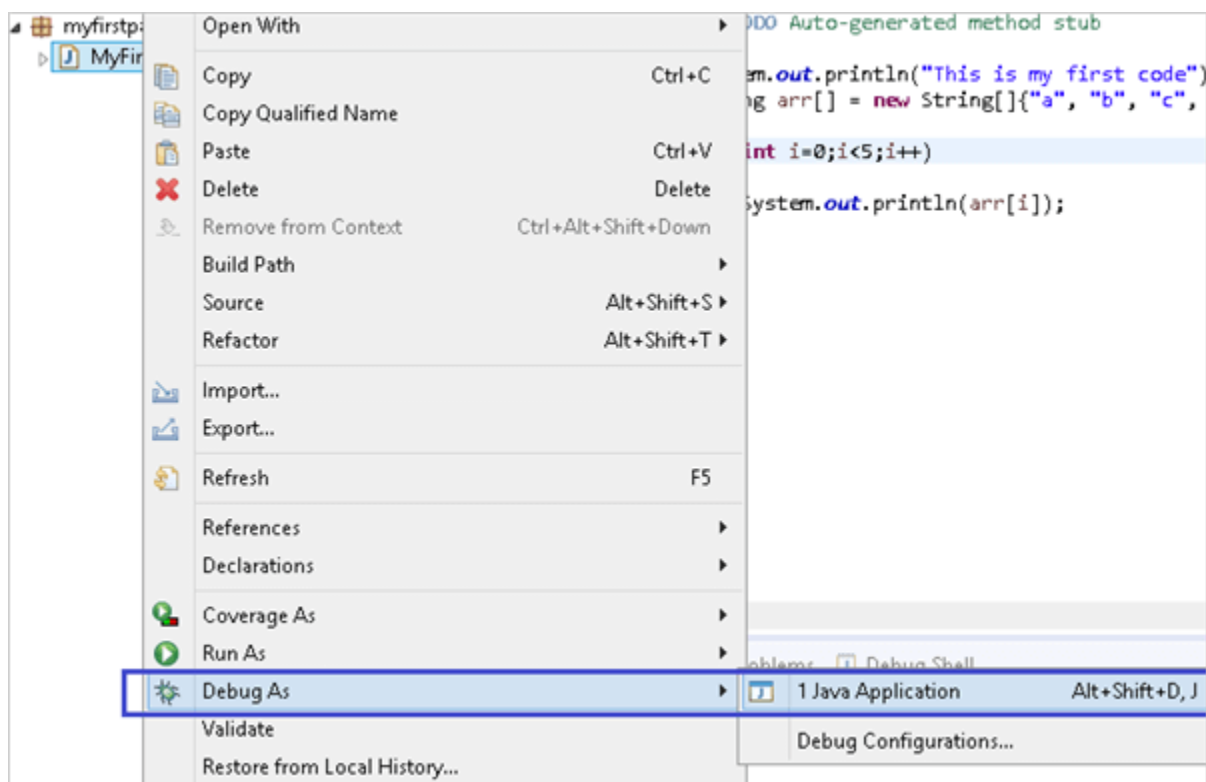
- If you right-click on this blue circle, you will get an option to toggle (add a breakpoint) or disable the breakpoint.
- You can also place a breakpoint by placing your cursor on the required code and press the keys "SHIFT + CTRL + B".
- Here we have placed the breakpoint at the start of the for a loop because if you see the exception that we got it is on the line inside this loop, thus you need to place this breakpoint at the code block where you are getting the error.

Running the Code in Debug

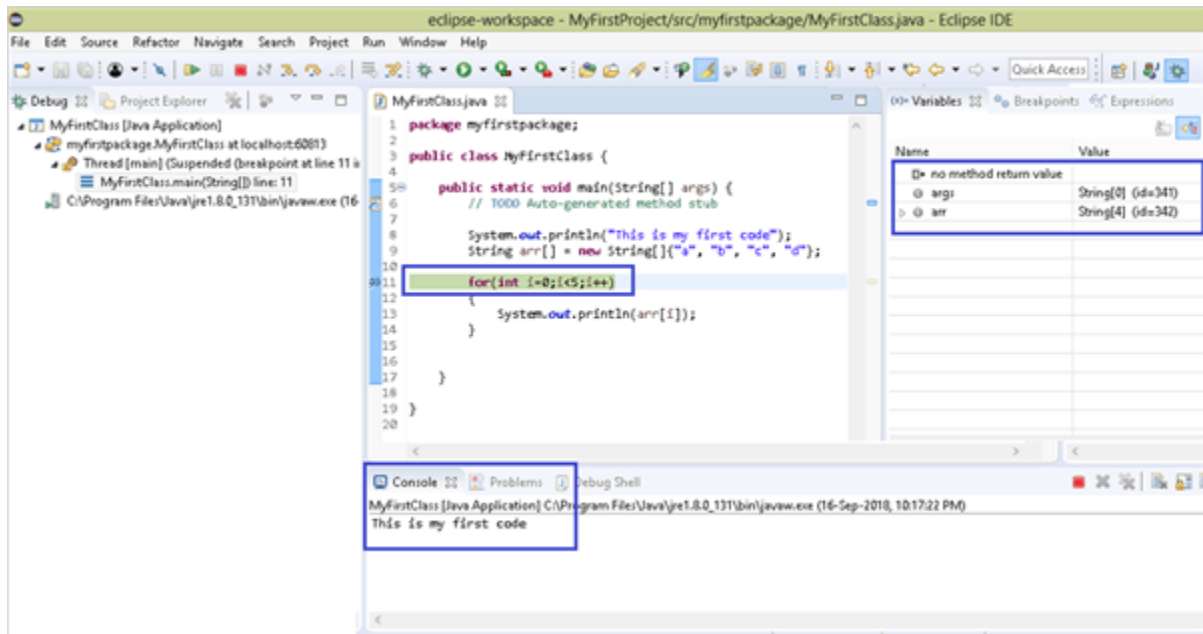
- Now, you have your program in debug mode and have placed the required breakpoints where we can run the code in debug mode.
- To run the code in debug mode you can do the following:
- Click on the icon shown below.



- Right-click on the class name from the project explorer and click on Debug As -> Java Application



- Once you run the application in debug mode, the following window will show up.



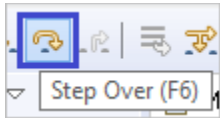
- You can see, the line on which the breakpoint is applied is highlighted and the execution of code has stopped at that point.
- You can verify this because the print statement before that line is executed on the console, however, the one inside the loop is not printed.
- You can also see the variables until this point is displayed in the variables section.
- If you expand the variable "arr", then you can see all the values in it.

(x)= Variables Breakpoints Expressions	
Name	Value
no method return value	
args	String[0] (id=341)
arr	String[4] (id=342)
[0]	"a" (id=343)
[1]	"b" (id=344)
[2]	"c" (id=345)
[3]	"d" (id=346)

- In this manner, we can see the value for any variable in the debug mode.

Now we need to manually enter the loop and we have certain key controls to do the same as shown below:

#2) Step over of F6: This option is also available as the icon shown below.



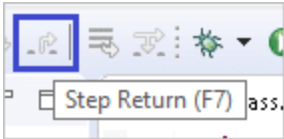
- Using this you can skip the debugger and execute the code.
- In this case, the code will execute normally till you keep hitting F6 and in the end, you will get the exception as you do while normally executing.

```
1 package myfirstpackage;
2
3 public class MyFirstClass {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7
8         System.out.println("This is my first code");
9         String arr[] = new String[]{"a", "b", "c", "d"};
10
11         for(int i=0;i<5;i++)
12         {
13             System.out.println(arr[i]);
14         }
15
16         System.out.println("This is my first code2");
17
18     }
19 }
20 }
21
```

<terminated> MyFirstClass [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe
This is my first code
a
b
c
d
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 4
at myfirstpackage.MyFirstClass.main(MyFirstClass.java:13)

#3) Step out or Step return or F7:

- This can be used when your code is in a method and is being called from another place.
- This key will finish the execution of the method being debugged and return to the code from where this method is being called.
- This can also be done by clicking on the below icon.



#4) Resume or F8:

- This option will tell the debugger to continue executing the program until the next breakpoint is reached.
- The icon for this is given below.

